ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №1 (Яндекс.Контест)

Выполнил:

Студент группы P3213

Якименко Владислав Игореич

Преподаватель:

Сорокин Артём Николаевич

Санкт-Петербург, 2026г.

Задачи из 1-го блока Яндекс.Контеста

Задача №1 «A. Агроном-любитель»

Городской школьник Лёша поехал на лето в деревню и занялся выращиванием цветов. Он посадил n*n*цветков вдоль одной длинной прямой грядки, и они успешно выросли. Лёша посадил множество различных видов цветков, i*i*-й от начала грядки цветок имеет вид ai*ai*, где ai*ai* — целое число, номер соответствующего вида в «Каталоге юного агронома».

Теперь Лёша хочет сделать фотографию выращенных им цветов и выложить ее в раздел «мои грядки» в социальной сети для агрономов «ВКомпосте». На фотографии будет виден отрезок из одного или нескольких высаженных подряд цветков.

Однако он заметил, что фотография смотрится не очень интересно, если на ней много одинаковых цветков подряд. Лёша решил, что если на фотографии будут видны три цветка одного вида, высаженные подряд, то его друзья — специалисты по эстетике цветочных фотографий — поставят мало лайков.

Помогите ему выбрать для фотографирования как можно более длинный участок своей грядки, на котором нет трех цветков одного вида подряд.

Формат ввода

В первой строке содержится целое число n*n* (1≤n≤2000001≤*n*≤200000) — количество цветов на грядке.

Во второй строке содержится n*n* целых чисел ai*ai* (1≤ai≤1091≤*ai*≤109), обозначающих вид очередного цветка. Одинаковые цветки обозначаются одинаковыми числами, разные — разными.

Формат вывода

Выведите номер первого и последнего цветка на самом длинном искомом участке. Цветки нумерются от 1 до n*n*.

Если самых длинных участков несколько, выведите участок, который начинается раньше.

Пример

Ввод	**Вывод**
6 5 6 6 6 23 9	3 6

Идея решения:

Рассматриваем цветки слева направо и поддерживаем текущий отрезок [left..right], в котором нет трёх одинаковых подряд.
На каждом шаге right — текущий цветок. Если a[right-2], a[right-1] и a[right] одинаковые, значит появилась запрещённая тройка, заканчивающаяся в right. Тогда сдвигаем начало: left = max(left, right-1).

Почему именно right-1: так в начале остаются максимум два одинаковых подряд, а длина текущего отрезка остаётся максимально возможной для этого right.

Дальше считаем длину текущего отрезка right-left+1. Если она больше лучшей найденной, обновляем глобальный ответ (начало и конец). Если длины равны, оставляем более ранний отрезок (меньший left), как требует условие.

После прохода по всем цветкам получаем максимальный по длине отрезок, в котором нет трёх одинаковых цветков подряд.

Сложность:

- Время: O(n)
- Память: O(1) (или O(n), если хранить массив входа)

Код:

```
/**
 * @author: Slava Yakimenko
 */

#include <algorithm>
#include <cmath>
#include <functional>
#include <iostream>
#include <map>
#include <numeric>
#include <queue>
#include <set>
#include <stack>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <vector>

using namespace std;
typedef long long ll;

#define fastIO()                 \
  do {                           \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);            \
    cout.tie(nullptr);           \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)

/*
 * HEADER END
 */

void solve() {
  ll n = 0;
```

```
  cin >> n;

  vector<ll> flowers(n);
  for (ll i = 0; i < n; ++i) {
    cin >> flowers[i];
  }

  ll left = 0;
  ll best_left = 0;
  ll best_right = 0;
  ll best_len = -INF;

  for (ll right = 0; right < n; ++right) {
    if (right >= 2 && flowers[right] == flowers[right - 1] &&
        flowers[right - 1] == flowers[right - 2]) {
      left = max(left, right - 1);
    }

    ll current_length = right - left + 1;

    if (current_length > best_len || (current_length == best_len && left < best_left)) {
      best_len = current_length;
      best_left = left;
      best_right = right;
    }
  }

  cout << best_left + 1 << ' ' << best_right + 1 << el;
}

int main() {
  fastIO();
  // freopen("tickets.in", "r", stdin);
  // freopen("tickets.out", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```

Задача №2 «B. Зоопарк Глеба»

Недавно Глеб открыл зоопарк. Он решил построить его в форме круга и, естественно, обнёс забором. Глеб взял вас туда начальником охраны. Казалось бы все началось так хорошо, но именно в вашу первую смену все животные разбежались. В зоопарке nn животных различных видов, также под каждый из видов есть свои ловушки. К сожалению некоторые животные враждуют с друг другом в природе (они обозначены разными буквами), а зоопарк обнесён забором и имеет форму круга. С помощью камер, удалось выяснить, где находятся все животные. Умная система поддержки жизнедеятельности зоопарка уже просканировала зоопарк и вывела id всех животных и ловушек в том порядке, в котором они видны из центра зоопарка. Получилось так, что все животные и все ловушки находятся на краю зоопарка. Вы хотите понять, могут ли животные прийти в свою ловушку так, чтобы их путь не пересекался с другими. Если да, также предъявите какую-нибудь из схем поимки животных.

Формат ввода

На вход подается строчка из 2·n2·n символов латинского алфавита, где маленькая буква - животное, а большая - ловушка. Размер строки не более 100000100000.

Формат вывода

Требуется вывести "Impossible", если решения не существует или "Possible", если можно вернуть всех животных в клетки. В случае если можно, то для каждой ловушки в порядке обхода требуется вывести индекс животного в ней.

Пример 1

Ввод	**Вывод**
ABba	Possible 2 1

Пример 2

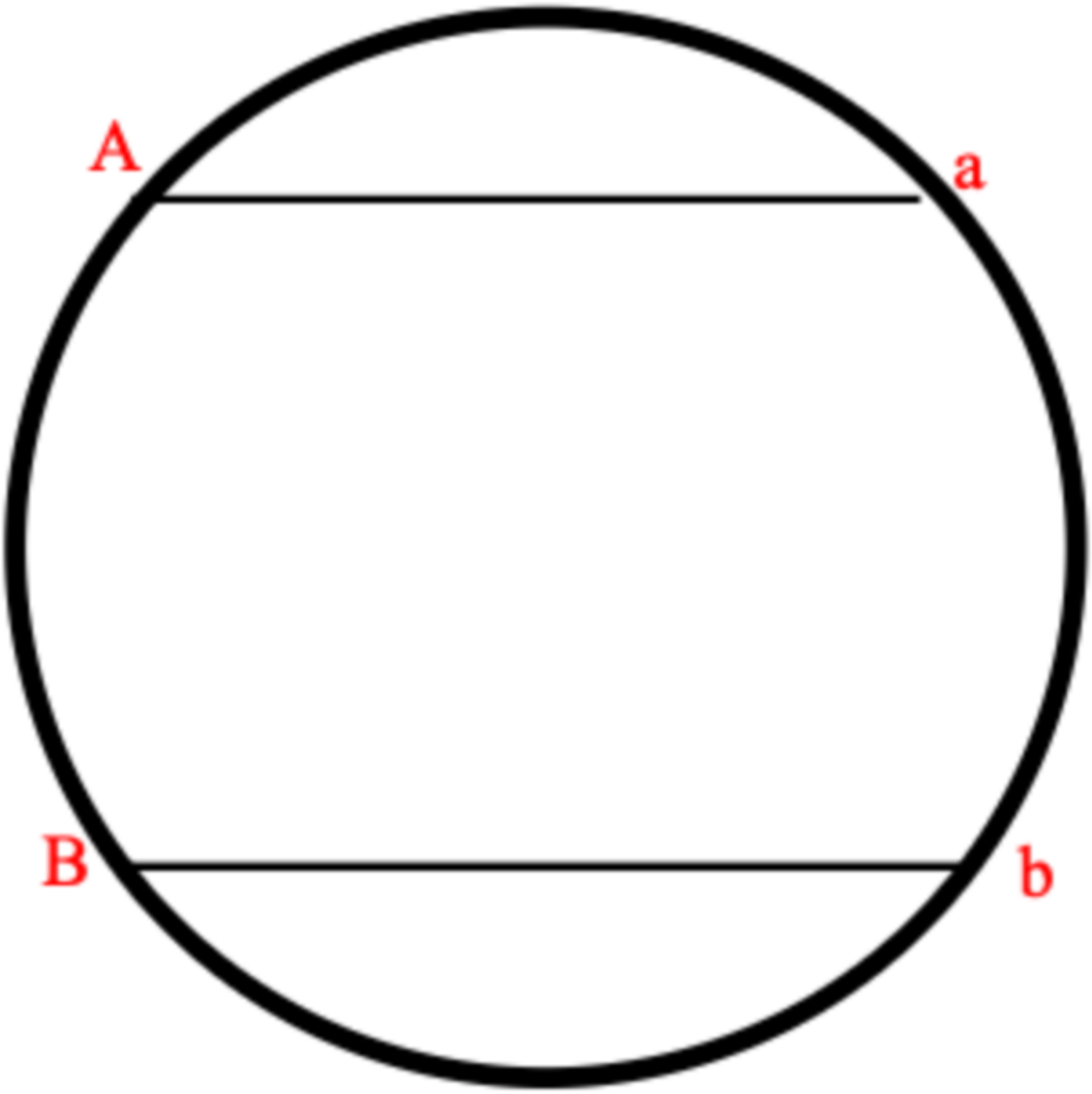
Ввод	**Вывод**
ABab	Impossible

Примечания

Первый пример:

Животное b идёт в ловушку B, а животное a ловится в ловушку A. Их пути не пересекаются,

поэтому



Второй пример:

Пути животных пересекаются, поэтому поймать их невозможно

Идея решения:

Обходим строку по кругу (в том порядке, как элементы стоят на границе) и используем стек позиций st.

1. Идём слева направо по строке s длины m.

2. Для текущей позиции i смотрим на вершину стека top = st.back():

- если s[top] и s[i] одна и та же буква без учёта регистра (same_kind), но в разных регистрах (opposite_case), то это пара

- такую пару удаляем из стека и фиксируем соответствие в answer (какое животное попало в какую ловушку)

- иначе текущую позицию кладём в стек: st.push_back(i)

3. Перед этим отдельно нумеруем:

- животных: animal_id[pos]

- ловушки: trap_id[pos]

Это позволяет при найденной паре записать нужный номер в answer.

4. После прохода:

- если стек не пуст, сделать непересекающееся сопоставление невозможно (Impossible);

- если стек пуст, сопоставление существует (Possible), выводим answer для ловушек по порядку.

Сложность:

- Время: O(n)
- Память: O(n)

Код:

```
/**
 * @author: Slava Yakimenko
 */

#include <algorithm>
#include <cctype>
#include <iostream>
#include <string>
#include <vector>

using namespace std;
typedef long long ll;

#define fastIO()                  \
  do {                            \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);             \
    cout.tie(nullptr);            \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)

/*
 * HEADER END
 */

void solve() {
  string s;
  cin >> s;
```

```
const ll m = static_cast<ll>(s.size());
vector<ll> animal_id(m, 0);
vector<ll> trap_id(m, 0);

ll animal_count = 0;
ll trap_count = 0;
for (ll i = 0; i < m; i++) {
  const unsigned char ch = static_cast<unsigned char>(s[i]);
  if (islower(ch)) {
    animal_id[i] = ++animal_count;
  } else {
    trap_id[i] = ++trap_count;
  }
}

vector<ll> answer(trap_count + 1, 0);
vector<ll> st;
st.reserve(m);

for (ll i = 0; i < m; ++i) {
  if (!st.empty()) {
    const ll top = st.back();
    const unsigned char top_ch = static_cast<unsigned char>(s[top]);
    const unsigned char cur_ch = static_cast<unsigned char>(s[i]);

    const bool same_kind = tolower(top_ch) == tolower(cur_ch);
    const bool opposite_case =
        (islower(top_ch) && isupper(cur_ch)) || (isupper(top_ch) && islower(cur_ch));

    if (same_kind && opposite_case) {
      st.pop_back();
      if (isupper(top_ch)) {
        answer[trap_id[top]] = animal_id[i];
      } else {
        answer[trap_id[i]] = animal_id[top];
      }
      continue;
    }
  }
  st.push_back(i);
}

if (!st.empty()) {
  cout << "Impossible" << el;
  return;
}

cout << "Possible" << el;
for (ll i = 1; i <= trap_count; ++i) {
  if (i > 1) {
    cout << ' ';
  }
  cout << answer[i];
```

```
  }
  cout << el;
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```

Задача №3 «C. Конфигурационный файл»

Вадим разрабатывает парсер конфигурационных файлов для своего проекта. Файл состоит из блоков, которые выделяются с помощью символов «`{`» — начало блока, и «`}`» — конец блока. Блоки могут вкладываться друг в друга. В один блок может быть вложено несколько других блоков.

В конфигурационном файле встречаются переменные. Каждая переменная имеет имя, которое состоит из не более чем десяти строчных букв латинского алфавита. Переменным можно присваивать числовые значения. Изначально все переменные имеют значение 00.

Присваивание нового значения записывается как `<variable>=<number>`, где `<variable>` — имя переменной, а `<number>` — целое число, по модулю не превосходящее $10^9 10^9$. Парсер читает конфигурационный файл построчно. Как только он встречает выражение присваивания, он присваивает новое значение переменной. Это значение сохраняется до конца текущего блока, а затем восстанавливается старое значение переменной. Если в блок вложены другие блоки, то внутри тех из них, которые идут после присваивания, значение переменной также будет новым.

Кроме того, в конфигурационном файле можно присваивать переменной значение другой переменной. Это действие записывается как `<variable1>=<variable2>`. Прочитав такую строку, парсер присваивает текущее значение переменной `variable2` переменной `variable1`. Как и в случае присваивания константного значения, новое значение сохраняется только до конца текущего блока. После окончания блока переменной возвращается значение, которое было перед началом блока.

Для отладки Вадим хочет напечатать присваиваемое значение для каждой строки вида `<variable1>=<variable2>`. Помогите ему отладить парсер.

Формат ввода

Входные данные содержат хотя бы одну и не более $10^5 10^5$ строк. Каждая строка имеет один из четырех типов:

- `{` — начало блока;
- `}` — конец блока;

- `<variable>=<number>` — присваивание переменной значения, заданного числом;
- `<variable1>=<variable2>` — присваивание одной переменной значения другой переменной. Переменные `<variable1>` и `<variable2>` могут совпадать.

Гарантируется, что ввод является корректным и соответствует описанию из условия. Ввод не содержит пробелов.

Формат вывода

Для каждой строки типа `<variable1>=<variable2>` выведите значение, которое было присвоено.

Пример

Ввод	Вывод

`a=b`
`b=123`
`var=b`
`b=-34`
`{`
`c=b`
`b=1000000000`
`d=b`
`{`
`a=b`
`e=var`
`}`
`}`
`b=b`	

Идея решения:

Обрабатываем вхождения построчно и храним текущие значения переменных в хеш-мапе v. Для поддержки вложенных блоков используем стек st, где каждый элемент - словарь изменений в текущем блоке (какие переменные и какие старые значения у них были до первого изменения в этом блоке)

Читаем строку s.

A. Если s == "{", открываем новый блок: добавляем пустой словарь в st
B. Если s == "}", закрываем блок:
    a. берём все сохранённые в st.back() старые значения;
    b. откатываем переменные в v к этим старым значениям;
    c. удаляем верхний словарь из стека

4. в ином случае это присваивание a=b или a=number:

- находим позицию "="
- a — левая часть, b — правая часть
- определяем, число справа или имя переменной

5. Перед любым изменением переменной a:если a ещё не сохранялась в текущем блоке st.back(), сохраняем её старое значение туда (или 0, если переменной не было)

6. Если справа число, то присваиваем a это число

7. Если справа переменная:

- берём текущее значение этой переменной (если её нет, это 0)
- печатаем это значение
- присваиваем его переменной a.

8. Все изменения живут только до конца текущего блока, потому что при "}" делается откат.

Сложность:

- Время: O(n)
- Память: O(n)

Код:

```
/**
 * @author: Slava Yakimenko
 */

#include <cctype>
#include <iostream>
#include <string>
#include <unordered_map>
#include <vector>

using namespace std;
typedef long long ll;

#define fastIO()                  \
  do {                            \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);             \
    cout.tie(nullptr);            \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)

/*
 * HEADER END
 */

void solve() {
  unordered_map<string, ll> v;
  vector<unordered_map<string, ll>> st;
  st.push_back(unordered_map<string, ll>());
  string s;

  while (cin >> s) {
    if (s == "{") {
      st.push_back(unordered_map<string, ll>());
      continue;
    }
    if (s == "}") {
```

```
      for (auto it = st.back().begin(); it != st.back().end(); ++it) {
        if (it->second == 0) {
          v.erase(it->first);
        } else {
          v[it->first] = it->second;
        }
      }
      st.pop_back();
      continue;
    }

    ll p = static_cast<ll>(s.find('='));
    string a = s.substr(0, p);
    string b = s.substr(p + 1);
    unsigned char c = static_cast<unsigned char>(b[0]);
    bool is_num = (c == '-') || isdigit(c);

    if (st.back().find(a) == st.back().end()) {
      if (v.find(a) == v.end()) {
        st.back()[a] = 0;
      } else {
        st.back()[a] = v[a];
      }
    }

    if (is_num) {
      ll x = stoll(b);
      if (x == 0) {
        v.erase(a);
      } else {
        v[a] = x;
      }
    } else {
      ll x = 0;
      if (v.find(b) != v.end()) {
        x = v[b];
      }
      cout << x << el;
      if (x == 0) {
        v.erase(a);
      } else {
        v[a] = x;
      }
    }
  }
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
```

```
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```

Задача №4 «D. Профессор Хаос»

В секретной лаборатории профессора Хаоса проходит эксперимент по выращиванию особо опасных бактерий. В начале первого дня эксперимента у Хаоса имеется aa особо опасных бактерий.

Каждый день эксперимента устроен следующим образом. Рано утром профессор достает из контейнера все свои бактерии и помещает их в инкубатор, где бактерии начинают делиться. Вместо каждой бактерии образуется bb новых бактерий.

После извлечения бактерий из инкубатора cc из них используются для проведения различных опытов и затем уничтожаются. Если после извлечения из инкубатора имеется менее cc бактерий, для проведения опытов используются все имеющиеся бактерии, и эксперимент заканчивается.

Оставшиеся бактерии в конце дня необходимо поместить в контейнер и продолжить использовать в эксперименте. Однако в контейнер можно поместить не более dd бактерий, поэтому если число оставшихся бактерий больше dd, то в контейнер помещаются dd бактерий, а остальные уничтожаются.

Теперь профессор Хаос хочет выяснить, сколько особо опасных бактерий будет у него в контейнере после kk-го дня эксперимента. Помогите ему найти ответ на этот вопрос.

Формат ввода

В единственной строке входного файла содержится пять целых чисел aa, bb, cc, dd и kk ($1 \le a, b \le 10001 \le a, b \le 1000$, $0 \le c \le 10000 \le c \le 1000$, $1 \le d \le 10001 \le d \le 1000$, $a \le d$ $a \le d$, $1 \le k \le 10181 \le k \le 1018$).

Формат вывода

Выведите одно число — количество бактерий у Хаоса к концу kk-го дня. Если эксперимент завершится в kk-й день или ранее, выведите число 0.

Пример 1

Ввод	Вывод
1 3 1 5 2	5

Пример 2

Ввод	Вывод
1 2 0 4 3	4

Пример 3

Ввод	**Вывод**
1 2 3 5 2	0

Идея решения:

Обозначим x_t как количество бактерий в контейнере в конце t-го дня. Тогда каждый день это просто переход по функции:

- размножение: y = x * b
- если y < c, то для опытов забирают всё и эксперимент сразу заканчивается, значит следующий x = 0
- иначе остаётся y – c
- если это число больше d, в контейнер помещают только d

Итого получаем функцию перехода f(x), и x_{t+1} = f(x_t), где x_0 = a

1. Мы не можем симулировать до k (k до 10^18), но состояние x всегда в диапазоне 0..d

2. Значит последовательность x_0, x_1, x_2, ... в конечном множестве обязательно войдёт в цикл

3. Храним:

- pos[val] — на каком шаге впервые встретилось значение val,

- seq — сами значения по шагам

4. Идём по переходам:

- пока текущее значение x ещё не встречалось, запоминаем его в pos и seq, переходим к f(x).

5. Когда встретили уже виденное значение:

- start = pos[x] — начало цикла

- len = текущий_шаг - start — длина цикла

6. Теперь ответ для шага k:

- если k меньше длины префикса до цикла, берём seq[k]

- иначе переходим внутрь цикла: индекс = start + (k - start) % len

7. Печатаем найденное значение.

Почему условие “если эксперимент завершился раньше, вывести 0” выполняется автоматически:

- состояние 0 считается поглощающее (go(0)=0)
- как только попали в 0, дальше всегда 0
- значит на любом более позднем дне ответ будет 0

Сложность:

- Время: O(d)
- Память: O(d)

Код:

```
/**
 * @author: Slava Yakimenko
 */

#include <iostream>
#include <vector>

using namespace std;
typedef long long ll;

#define fastIO()                  \
  do {                            \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);             \
    cout.tie(nullptr);            \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)

/*
 * HEADER END
 */

void solve() {
  ll a = 0, b = 0, c = 0, d = 0, k = 0;
  cin >> a >> b >> c >> d >> k;

  auto go = [&](ll x) -> ll {
    ll y = x * b;
    if (y < c) {
      return 0;
    }
    ll r = y - c;
    if (r > d) {
      return d;
    }
    return r;
  };

  vector<ll> pos(d + 1, -1);
  vector<ll> seq;

  ll x = a;
  ll day = 0;
  while (pos[x] == -1) {
```

```
    pos[x] = day;
    seq.push_back(x);
    x = go(x);
    ++day;
  }

  ll start = pos[x];
  ll len = day - start;

  ll ans = 0;
  if (k < static_cast<ll>(seq.size())) {
    ans = seq[k];
  } else if (k < start) {
    ans = seq[k];
  } else {
    ll id = start + (k - start) % len;
    ans = seq[id];
  }
  cout << ans << el;
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```